abstraction.py Detailed Documentation

Mathematical Report: State Encoding, Rotation, and Value Ranges

Source file: c:\Users\PRABAL YADAV\Desktop\machine learning iim\pokerbot bestmodelcopy 6people\abstraction.py

Generated: 2025-12-24 15:01

This report documents every computation used by abstraction.py and focuses on encode_state: required inputs, expected ranges, and the positional mapping that lets the model infer SB, BB, and the player to act.

Note: abstraction.py contains an older commented block near the top. The active implementation begins after the second file overview and the real imports.

Executive summary

encode_state returns a 1D torch.FloatTensor whose length depends on the number of seats (N) and the action history length (K = ACTION_SEQ_LEN). The exact length is 30 + 8N + 7K. For N=6 and K=6, the vector length is 120.

The model learns seat position via two mechanisms: (1) hero_position_one_hot relative to the button and (2) explicit hero_is_sb and hero_is_bb flags. The player to act is represented in the per-seat features with a to_act_flag, rotated so the hero is always the first seat in the per-seat block.

If any normalized feature falls outside its expected range, the model can misbehave. This report lists expected ranges and invariants that should hold for correctness.

GameState and rotation logic

Seat indices are integers 0..N-1. The button rotates each hand. In SimpleHoldemEnv, button starts at a random seat and increments by 1 each new_hand(). In CashSession (used by gui_play), the button rotates only among live seats so busted players are skipped.

Button, SB, BB assignment

For N>2: SB is next live seat clockwise from button, BB is next live seat clockwise from SB.
For N=2: SB and button are the same seat, BB is the other seat.

```
N=6 example with button=2
Seat index:   0  1  2  3  4  5
Role:        HJ CO BTN SB BB UTG
hero_pos = (player - button) mod 6
hero_pos==0 -> BTN, 1 -> SB, 2 -> BB, 3 -> UTG, 4 -> HJ, 5 -> CO
```

Action order and to_act

Preflop to_act is the first live seat clockwise from the BB. Postflop to_act is the first live seat clockwise from the button. The engine uses _find_next_player with include_start=True to skip folded or empty-stack seats.

```
Preflop: to_act = next_live(bb + 1)
Postflop: to_act = next_live(button + 1)
```

If no valid player is found, to_act is set to -1. In encode_state, this yields all to_act_flag values = 0.

Card representation

Cards are integers 0..51. Ranks are 2..14. Suits are 0..3 with the mapping 0=spades, 1=hearts, 2=diamonds, 3=clubs.

```
card_rank(card) = (card % 13) + 2
card_suit(card) = card // 13
Examples: card=0 -> 2 of spades, card=51 -> A of clubs
```

_card_0_51_to_treys converts 0..51 into Treys integer codes used by the evaluator.

Hand evaluation and strength

evaluate_5card and evaluate_7card are wrappers around Treys Evaluator. Treys returns smaller scores for stronger hands; abstraction.py negates the value so higher is better.

normalized_strength(hole, board) returns a float in [0,1]. Preflop uses a simple rank heuristic. Postflop uses a LUT (hand_strength_lut.pkl) that maps evaluate_7card scores to percentiles.

```
Preflop heuristic: strength = (hi_rank + lo_rank) / (2 * 14)
Postflop: strength = percentile(score) from LUT
```

LUT values are built from random 7-card samples. If the LUT file does not exist, import will generate it, which can take time.

Board texture and draw signals

_board_texture(board) returns (highness, connectedness, suitedness). highness is max rank / 14. connectedness is based on the longest straight window on the board. suitedness is the max suit count divided by board length.

_has_flush_draw checks if any suit occurs at least 4 times in the combined cards. _has_straight_draw checks if any 5-card window has at least 4 ranks present. _hand_class returns the Treys class string for a 5+ card set.

Strength buckets

Buckets are 3-element one-hot vectors. They provide coarse information about strength relative to the street.

```
_preflop_bucket: [strong, medium, weak]
_flop_bucket: [air, draw, made]
_turn_river_bucket: [weak, medium, strong]
_street_strength_bucket: dispatches based on street
```

encode_state: required inputs and expected ranges

encode_state expects a GameState-like object. All list fields must be length N (num_players). Card lists must contain valid 0..51 ids.

Field	Type	Expected range or format
street	int	0..3
num_players	int	>=2
button_player	int	0..N-1
sb_player	int	0..N-1
bb_player	int	0..N-1
pot	float	>=0
current_bet	float	>=0 and >= max(contrib)
contrib	List[float]	each >=0, length N
stacks	List[float]	each >=0, length N
initial_stacks	List[float]	each >0, length N
folded	List[bool]	length N
players_acted	List[bool]	length N
to_act	int	-1 or 0..N-1
last_aggressor	int	-1 or 0..N-1
board	List[int]	length 0,3,4,5; cards 0..51
hole	List[List[int]]	N lists of length 2; cards 0..51
action_seq	List[Tuple]	list of (player, action, size_norm)

If any index is out of range, encoding still runs but the model receives invalid signals. Treat

these as data errors.

encode_state: feature layout

Total length = 30 + 8N + 7K, where K = ACTION_SEQ_LEN. Layout (in order):

```
1) street_one_hot (4)
2) hero_position_one_hot (N)
3) public scalars (10)
4) strength_bucket (3)
5) range and pressure features (9)
6) per-seat features (N * 7)
7) hole_feats (4)
8) action_seq (K * 7)
```

Street and position

street_one_hot is a 4-length one-hot for street in {0,1,2,3}. hero_position_one_hot is an N-length one-hot computed as hero_pos = (player - button_player) mod N.

This makes position relative to the button explicit. SB and BB are additionally encoded by hero_is_sb and hero_is_bb flags in the scalars block.

Public scalars (10 values)

```
pot_norm = pot / (ref_stack * max(2, N))
ref_stack = max(initial_stacks) if available else STACK_SIZE
to_call = max(0, current_bet - contrib[hero])
to_call_norm = to_call / ref_stack
pot_after_call_norm = (pot + to_call) / (ref_stack * max(2, N))
curr_bet_norm = current_bet / ref_stack
spr_raw = stacks[hero] / max(1, pot)
spr_norm = min(spr_raw, 20) / 20
hero_is_sb in {0,1}
hero_is_bb in {0,1}
last_agg_present in {0,1}
last_agg_rel = ((hero - last_aggressor) mod N) / (N-1)
```

Expected ranges: pot_norm, to_call_norm, pot_after_call_norm, curr_bet_norm in [0, ~1]. spr_norm in [0,1]. last_agg_rel in [0,1].

Strength and range features

board_str is normalized_strength([], board) in [0,1]. strength_bucket is a 3-length one-hot based on street and hand class. Range/pressure features are all in [0,1].

```
range_advantage = base_adv if hero was last aggressor else 1 - base_adv
base_adv = 0.6 * board_highness + 0.4 * dryness
dryness = 0.5*(1-connectedness) + 0.5*(1-suitedness)
hero_polarized = 1 if hero last aggressor, size_norm>=0.6, spr_raw<=3
villain_polarized = 1 if villain last aggressor, size_norm>=0.6, spr_raw<=3
fold_equity_proxy = min(villain_to_call / villain_stack, 1)
pot_pressure = min(villain_to_call / max(1,pot), 4) / 4
```

Blocker flags (top pair, nut flush, high card on flush board) are 0 or 1. line_consistency is a ratio in [0,1].

Per-seat features

Per-seat features are rotated so the hero is first. For each seat, the 7 values are:

```
stack_norm, contrib_norm, to_call_norm, folded_flag, all_in_flag, acted_flag, to_act_flag
```

Ranges: stack_norm, contrib_norm, to_call_norm are >=0 and typically <=1. The flags are 0 or 1. to_act_flag is 1 for the player to act, otherwise 0.

Rotation: order = [hero, hero+1, hero+2, ...] mod N. This makes the model invariant to absolute seat numbers.

Hole card features

encode_hole_cards returns [hi_rank_norm, lo_rank_norm, suited, pair] with each value in [0,1].

Action sequence

action_seq entries are tuples (player, action, size_norm) added by poker_env._record_action. size_norm is min(invested / pot_before, 4) / 4 and is therefore in [0,1].

```
For each of K actions (latest first):
actor_offset, is_fold, is_check, is_call, is_raise_small, is_raise_big, size_norm
actor_offset = ((actor - hero) mod N) / (N-1)
```

ACTION_RAISE_MEDIUM and ACTION_ALL_IN are grouped as is_raise_big.

Expected value ranges and sanity checks

The following checks help verify correctness. Any deviation may indicate a bug in the environment state or in the hand update logic.

```
Check                                          Expected
-------------------------------------------------------------------------------
Sum(street_one_hot)                             == 1
Sum(hero_position_one_hot)                     == 1
hero_is_sb + hero_is_bb                         in {0,1} (heads-up can be 1 for SB=BTN)
current_bet >= max(contrib)                    True
pot >= sum(contrib)                             True
0 <= spr_norm <= 1                               True
0 <= size_norm <= 1                              True
If to_act is -1                                 all to_act_flag == 0
If folded[i] == True                            player may still have contrib>0 (ok)
```

If pot_norm or to_call_norm regularly exceed 1, ref_stack may be too small or initial_stacks may be missing. If to_act_flag is set for multiple seats, the state is inconsistent.

Worked example

Assume N=6, hero=0, button=2, street=flop, pot=12, current_bet=4, contrib[hero]=2, stacks[hero]=96, initial_stacks=100, board size=3.

```
hero_pos = (0 - 2) mod 6 = 4
hero_position_one_hot[4] = 1
ref_stack = 100
pot_norm = 12 / (100*6) = 0.020
to_call = 4 - 2 = 2
to_call_norm = 0.02
pot_after_call_norm = 14 / 600 = 0.0233
curr_bet_norm = 0.04
spr_raw = 96 / 12 = 8.0
```

```
spr_norm = 8 / 20 = 0.40
```

These values are inserted into the vector in the order described above, followed by the remaining per-seat features, hole_feats, and action_seq encoding.

Common data errors and their effects

1) Wrong button_player or hero_position_one_hot. Effect: the model misinterprets position and can overfold or overaggress in the wrong spots.

2) Incorrect to_act. Effect: action order is confused and the model may learn illegal or nonsensical timing patterns.

3) Missing initial_stacks. Effect: normalization scales are off; pot_norm and to_call_norm can be too large or too small.

4) action_seq not updated. Effect: loss of line information; the model underestimates aggression and bet sizing patterns.

Appendix: parameter values from config.py

The following defaults influence expected ranges. If you change them, update your expectations accordingly.

```
STACK_SIZE = 200.0
SMALL_BLIND = 1.0
BIG_BLIND = 2.0
NUM_PLAYERS = 6
ACTION_SEQ_LEN = 6
```